HAL open science

Verified Path Indexing

Mohamed Chaabani, Simon Robillard

▶ **To cite this version:**

Mohamed Chaabani, Simon Robillard. Verified Path Indexing. CADE 30 - 30th International Conference on Automated Deduction, Jul 2025, Stuttgart, Germany. In press. lirmm-05239240

HAL Id: lirmm-05239240

https://hal-lirmm.ccsd.cnrs.fr/lirmm-05239240v1

Submitted on 4 Sep 2025

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.

Verified Path Indexing

Mohamed Chaabani[1,2] and Simon Robillard[1] ✉

[1] LIRMM, Université de Montpellier & CNRS, Montpellier, France
[2] LIMOSE, University of Boumerdès, Algeria
firstname.lastname@lirmm.fr

Abstract. The indexing of syntactic terms is a key component for the efficient implementation of automated theorem provers. This paper presents the first verified implementation of a term indexing data structure, namely a formalization of path indexing in the proof assistant Isabelle/HOL. We define the data structure, maintenance operations, and retrieval operations, including retrieval of unifiable terms, instances, generalizations and variants. We prove that maintenance operations preserve the invariants of the structure, and that retrieval operations are sound and complete.

1 Introduction

In the realm of program verification, automated deduction algorithms hold a special place. They are a cornerstone of automated program verification, and as such, they are also a natural target for verification. Indeed, verified automated deduction programs would be useful not only as trusted implementations, but also as witnesses to the usability of program verification. After all, if the program verification community hopes to convince industrial actors to adopt and use its techniques for software development, it should first be able to do the same.

However, automated deduction algorithms present a particularly difficult challenge for formal verification, as they rely on a rich and complex meta-theory, as well as on optimized implementations and data structures that are crucial for performance. Many efforts have been undertaken to formalize the meta-theory of CDCL SAT solving [1], first-order resolution [10], or saturation theorem proving [13]. These formalizations focus on high-level properties of the calculi used in automated reasoning tools. They are typically expressed in a relational rather than functional style, and thus do not directly yield executable programs. The second step toward formal verification of automated deduction techniques is to formalize and verify the algorithms and data structures used to implement the calculi. State-of-the-art (non-verified) automated deduction tools employ advanced algorithms and data structures. In order to obtain verified executable automated reasoning tools with a decent performance, these implementation-oriented details have to be verified as well. For example, the verified SAT solver IsaSAT [2] relies on a two-watched-literal data structure for unit propagation and conflict detection, similarly to its unverified competitors.

Automated reasoning tools for first-order logic are significantly more complex than those for propositional logic, and for now, a verified executable prover for

first-order logic has yet to be implemented. One key ingredient for performance in automated reasoning in first-order logic is efficient term indexing. Virtually all modern reasoning techniques for first-order logic are based on term unification, and therefore require a way to find unifiable terms among a set of candidates. The term unification problem is quasi-linear in the size of the candidate terms, but since theorem provers can handle millions of candidates, it is not feasible to iterate through pairs of candidates and attempt to unify them. Instead, theorem provers rely on specialized data structures to store terms and retrieve those that are unifiable with a given query term.

This paper presents the formalization[3], in the Isabelle/HOL proof assistant [8,7], of path indexing [9,12], a term indexing technique that relies on a trie to store candidate terms. After an introduction to Isabelle/HOL (Section 2) and its notations, and some preliminary notions on terms and term indexing (Section 3), this paper presents the formalization of path indexing (Section 4). We define the data structure, its invariants, maintenance and retrieval operations, and we sketch the correctness proofs for the different operations. This work represents the first verified implementation of a term indexing technique. Although path indexing has been studied from the perspective of implementation and performance [6,3], this is also, to our knowledge, the first detailed proof of correctness for this technique.

2 Proofs and Programs in Isabelle/HOL

Isabelle/HOL is an interactive proof assistant for higher-order logic. It offers powerful features that simplify and automate the proof process: its rewriting engine allows substituting terms with equivalents, the axiom of substitution allows replacing any subterm of the proof with an equal term, and the rule of extensionality allows proving the equality of two functions by proving that they produce the same result for each argument.

Isabelle allows the definition of inductive datatypes. For example, the type of lists can be defined as follows:

```
datatype 'a list =
    Nil ([])
  | Cons (hd: 'a) (tl: 'a list) (infix # 65)
```

This is a polymorphic definition with a type parameter *'a*, that declares *Nil* and *Cons* as the two constructors of the type. In addition, it also defines two partial functions *hd* and *tl* to access the two arguments of lists built from *Cons*, and adds notations [] and # for the constructors, the second being an infix operator. Predicates can likewise be defined inductively, for example list membership can be defined as:

```
inductive ListMem :: 'a ⇒ 'a list ⇒ bool where
  ListMem x (x # xs)
| ListMem x xs ⟹ ListMem x (y # xs)
```

[3] Available at https://seafile.lirmm.fr/f/d76bb08ee2cf4cacb823/

In this definition, Isabelle's metalogical implication, $\Longrightarrow$, is used to set the condition for the second case. Within statements, the object-logic implication, denoted $\longrightarrow$ must be used instead. The first notation is not to be confused with the notation for function types, $\Rightarrow$.

We can also define functions such as the following

fun *nth* :: *'a list* ⇒ *nat* ⇒ *'a* **where**
  nth [] *n* = *undefined*
| *nth* (*x* # *xs*) *n* = (*case n of 0* ⇒ *x* | *Suc m* ⇒ *nth xs m*)

that performs pattern matching on a value of *nat*, the type of natural numbers with constructors *0* and *Suc*, in order to return the *nth* element of the list. The Isabelle library use the infix notation *l* ! *i* for *nth l i*, and we will use this notation in the following. Functions can be partial (non-exhaustive), but their termination must be proven. In simple cases such as the structural recursion above, this obligation is discharged automatically.

Isabelle also features type definitions that identify a new type with a subset of an existing type. For example finite maps are defined as

typedef (*'a*, *'b*) *fmap* = {*m*. *finite* {*a*. *m a* ≠ *None*} } :: (*'a* ⇒ *'b option*) *set*

the subset of partial functions (*a* ⇒ *'b option*) that are defined for a finite number of values. Such a definition requires a proof that the newly defined type is inhabited. It also introduces coercion functions. The Lifting package allows functions on the supertype to be conveniently adapted to the subtype, provided that the necessary proofs of membership are given. We will use finite maps in this formalization, and in particular we rely on the following notations: *fmempty* stands for the map that associates every value to *None*, *fmdom' m* and *fmran' m* are the domain and range of a map *m*, *m* $$ *v* is the evaluation of *m* at value *v*, while *m* $$! *k* is the evaluation of *m* at value *v* provided that *v* ∈ *fmdom' m* (i.e., $$ is a total function that returns an element of *'b option*, while $$! returns an element of *'b* and may be undefined).

3 Preliminaries

This formalization relies on a pre-existing definition of syntactic terms, available in the Archive of Formal Proofs (AFP), the main repository of Isabelle formalizations. The definition is the following:

datatype (*'f*,*'v*) *term* =
  Var 'v
| *Fun 'f* (*'f*,*'v*) *term list*

This definition is parameterized by a type *'f* of function symbols, and a type *'v* of variables. Since there are no constraints on the number of arguments that a function symbol is applied to, symbols in terms are uniquely identified by a pair of type (*'f* × *nat*), the second element being the arity. For example, the term *Fun s* [*Fun s* []] is valid but features two distinct symbols (*s*, *1*) and (*s*, *0*).

A *term index* is a data structure that stores a set S of terms, and that is able, given a query term t, to retrieve all the terms $s \in S$ that satisfy $R(s,t)$, for some R called the retrieval condition. In practice, there are four retrieval conditions that are of particular interest, all related to unification problems ($t \cdot \sigma$ denotes the application of a substitution σ to a term t):

definition *term-unification* :: $('f, 'v)$ *term* $\Rightarrow$ $('f, 'v)$ *term* $\Rightarrow$ *bool* **where**
term-unification t1 t2 $= (\exists\, \sigma :: ('f, 'v)$ *subst*. $t1 \cdot \sigma = t2 \cdot \sigma)$

definition *term-instantiation* :: $('f, 'v)$ *term* $\Rightarrow$ $('f, 'v)$ *term* $\Rightarrow$ *bool* **where**
term-instantiation t1 t2 $= (\exists\, \sigma :: ('f, 'v)$ *subst*. $t1 = t2 \cdot \sigma)$

definition *term-generalization* :: $('f, 'v)$ *term* $\Rightarrow$ $('f, 'v)$ *term* $\Rightarrow$ *bool* **where**
term-generalization t1 t2 $=$ *term-instantiation t2 t1*

definition *term-variant* :: $('f, 'v)$ *term* $\Rightarrow$ $('f, 'v)$ *term* $\Rightarrow$ *bool* **where**
term-variant t1 t2 $= (\exists\, \sigma :: ('f, 'v)$ *subst*. $(\forall\, t \in$ *subst-range* σ. *is-Var* $t) \wedge t1 \cdot \sigma = t2)$

For example, retrieval of unifiable terms is used to implement resolution rules or to find closing substitutions in analytical tableaux. Resolution theorem provers rely on retrieval of instances to implement forward subsumption and demodulation, and retrieval of generalizations for backward subsumption and demodulation. Aside from automated deduction, other applications that perform computations over terms can benefit from term indexing, including functional programming, logic programming and deductive databases. For an overview of these applications, and what retrieval conditions they use, we refer the reader to the work of Sekar et al. [11].

Like many term indexing techniques, path indexing is a so-called *imperfect filter*, i.e., its retrieval conditions are broader that the aforementioned relations. Consequently, the set that is returned for a given query is a superset of the intended result, and must be filtered using a standard unification algorithm (which is also an opportunity to construct the unifiers, for applications that need that information). More specifically, the retrieval conditions used in path indexing ignores the *non-linearity* of terms, i.e., the fact that a single variable may occur multiple times in the query term and/or indexed terms, and the resulting contraints w.r.t. term unification. Instead, it relies on the following relations as retrieval conditions:

inductive *linear-unification* :: $('f, 'v)$ *term* $\Rightarrow$ $('f, 'v)$ *term* $\Rightarrow$ *bool* **where**
linear-unification (*Var x*) *t*
| *linear-unification t* (*Var x*)
| *length args1* $=$ *length args2* $\Longrightarrow$
$(\forall\, i.\ i <$ *length args1* $\longrightarrow$ *linear-unification* (*args1* ! *i*) (*args2* ! *i*)) $\Longrightarrow$
linear-unification (*Fun f args1*) (*Fun f args2*)

inductive *linear-instance* :: $('f, 'v)$ *term* $\Rightarrow$ $('f, 'v)$ *term* $\Rightarrow$ *bool* **where**
linear-instance t (*Var x*)
| *length args1* $=$ *length args2* $\Longrightarrow$

$(\forall i.\ i < \mathit{length\ args1} \longrightarrow \mathit{linear\text{-}instance}\ (\mathit{args1}\ !\ i)\ (\mathit{args2}\ !\ i)) \Longrightarrow$
$\mathit{linear\text{-}instance}\ (\mathit{Fun\ f\ args1})\ (\mathit{Fun\ f\ args2})$

definition *linear-generalization t e = linear-instance e t*

inductive *linear-variant* :: $('f,\ 'v)$ *term* $\Rightarrow$ $('f,\ 'v)$ *term* $\Rightarrow$ *bool* **where**
linear-variant (*Var x*) (*Var y*)
| *length args1* = *length args2* $\Longrightarrow$
$(\forall i.\ i < \mathit{length\ args1} \longrightarrow \mathit{linear\text{-}variant}\ (\mathit{args1}\ !\ i)\ (\mathit{args2}\ !\ i)) \Longrightarrow$
linear-variant (*Fun f args1*) (*Fun f args2*)

We can check that these relations are necessary conditions for their non-linear counterparts, and the set returned by the index for a given query is in fact a superset of the intended result, e.g.:

lemma *linear-unification-necessary-condition*:
term-unification t1 t2 $\Longrightarrow$ *linear-unification t1 t2*

Since path indexing ignores the specific names of variables, we introduce the following datatype to describe the symbols that it operates on, together with a function that extracts said symbol from a term:

datatype $'f$ *symbol* =
Star
| *F* $'f$ *nat*

fun *to-symbol* :: $('f,\ 'v)$ *term* $\Rightarrow$ $'f$ *symbol* **where**
to-symbol (*Var x*) = *Star*
| *to-symbol* (*Fun f ts*) = *F f* (*length ts*)

Path indexing relies on the *path strings* (simply called *paths* in the following) of terms to index them. A path string is a list of symbol-number pairs that corresponds to a path from the root to a node in the tree structure of a term.

type-synonym $'f$ *tpath* = ($'f$ *symbol* $\times$ *nat*) *list*

Although this is not specified in the type itself, in a valid path, each symbol s can only be paired with a number i such that $i <$ *symbol-arity* s, except if *symbol-arity* s = *0*, in which case, i is *0* by convention.

We may use paths to navigate a term, e.g., *subterm-at-path t p t'* holds if and only if t' is the subterm found by following the path p from the root of t:

inductive *subterm-at-path* :: $('f,'v)$ *term* $\Rightarrow$ $'f$ *tpath* $\Rightarrow$ $('f,'v)$ *term* $\Rightarrow$ *bool* **where**
subterm-at-path t [] *t*
| $i <$ *length fargs* $\Longrightarrow$ *fargs* $\neq$ [] $\Longrightarrow$ *subterm-at-path* (*fargs* ! i) *p t* $\Longrightarrow$
subterm-at-path (*Fun f fargs*) ((*F f* (*length fargs*), i) # *p*) *t*

In particular, we are interested in the paths to leaves (constant or variables) of a term. A string p is a path to a leaf of a term t if and only if it satisfies *path-of-term t p*, defined as follows:

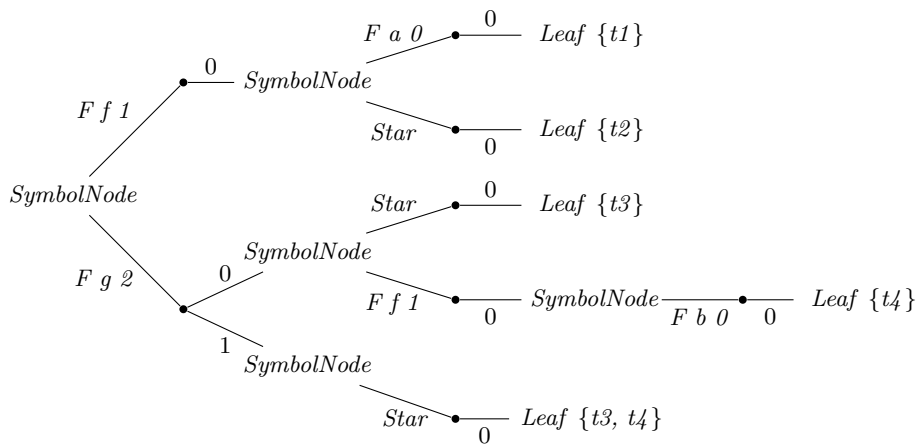

Fig. 1: The path indexing trie that stores the four following terms:
- $t1 \equiv Fun\ f\ [Fun\ a\ []]$,
- $t2 \equiv Fun\ f\ [Var\ x\ ]$,
- $t3 \equiv Fun\ g\ [Var\ x,\ Var\ y]$ and
- $t4 \equiv Fun\ g\ [Fun\ f\ [Fun\ b\ []],\ Var\ x]$.

inductive *path-of-term* :: $('f,'v)$ *term* $\Rightarrow$ $'f$ *tpath* $\Rightarrow$ *bool* **where**
  path-of-term (*Var x*) [(*Star*, *0*)]
| *path-of-term* (*Fun f* []) [(*F f 0*, *0*)]
| $ts \neq [] \Longrightarrow i < length\ ts \Longrightarrow$ *path-of-term* ($ts\ !\ i$) $p \Longrightarrow$
    path-of-term (*Fun f ts*) ((*F f* (*length ts*), *i*) # *p*)

When we mention the *paths of a term t*, we refer exclusively to those paths p that satisfy *path-of-term t p*. For example, given the variable x and the symbols a, f, and g, the term *Fun f* [*Fun g* [*Var x*], *Fun a*] has two paths, namely [(*F f 2, 0*), (*F g 1, 0*), (*Star, 0*)] and [(*F f 2, 1*), (*F a 0, 0*)].

4 Path Indexing

4.1 Structure and invariants

In path indexing, terms are stored in a *trie*, a tree structure in which edges are decorated with symbol-number pairs. We define this structure as follows:

datatype (*plugins del*: *size*) $('f,'v)$ *trie* =
  Leaf $('f,'v)$ *term set*
| *SymbolNode* (*children*: (($'f$ *symbol*, (*nat*, ($'f$, $'v$) *trie*) *fmap*) *fmap*))

Each node stores its children in nested *fmap*s, with the outer maps indexed by $'f$ *symbol*s and the inner maps indexed by *nat*s. The terms themselves are stored in the leaves of the trie. Figure 1 gives an example of a path indexing trie.

Because the recursive structure of this datatype is obfuscated by the maps, the *size* function that is automatically generated by Isabelle as a termination measure for the datatype is of no help in this case. Instead, we deactivate this feature and define our own function, based on the depth of the trie, prove its termination and register it as a termination measure. The proof of termination for functions that are defined by structural induction can subsequently be discharged automatically. This is, for example, the case with the following function, that collects all the terms stored in the trie:

fun *to-set* :: $('f,'v)$ *trie* $\Rightarrow$ $('f,'v)$ *term set* **where**
  to-set (*Leaf* x) = x
| *to-set* (*SymbolNode* m) = $\bigcup$ (*to-set* ‘ $\bigcup$ (*fmran'* ‘ (*fmran'* m)))

where the backquote ‘ denotes the application of a function to all the elements of a set.

Just as we use paths to navigate the tree structure of terms, we can use them to navigate tries. In particular, the trie *tr2* is the subtree of the trie *tr1* found by following the path p along the edges if and only if *subtrie-at-path tr1 p tr2*, defined as follows:

inductive *subtrie-at-path* :: $('f,'v)$ *trie* $\Rightarrow$ *'f tpath* $\Rightarrow$ $('f,'v)$ *trie* $\Rightarrow$ *bool* **where**
  subtrie-at-path tr [] *tr*
| *m1* \$\$ *s* = *Some m2* $\Longrightarrow$ *m2* \$\$ *i* = *Some tr* $\Longrightarrow$
    subtrie-at-path tr ps tr0 $\Longrightarrow$ *subtrie-at-path* (*SymbolNode m1*) ((s, i) # *ps*) *tr0*

The main invariant of a path indexing trie is that each term is stored in the leaves that are found at the end of paths of that term:

definition *sound-storage* :: $('f,\ 'v)$ *trie* $\Rightarrow$ *bool* **where**
  sound-storage tr =
    ($\forall$ *t p*. ($\exists$ *s*. *subtrie-at-path tr p* (*Leaf* (*insert t s*))) $\longrightarrow$ *path-of-term t p*)

where *insert* is the set insertion function. Since a term is stored in multiple leaves (as many as the constants and variables in that term), we must define a second invariant that ensures that any term present in the trie is present in all the relevant leaves of that trie:

definition *complete-storage* :: $('f,\ 'v)$ *trie* $\Rightarrow$ *bool* **where**
  complete-storage tr =
($\forall$ *t*$\in$*to-set tr*. $\forall$ *p*. *path-of-term t p* $\longrightarrow$ ($\exists$ *s*. *subtrie-at-path tr p* (*Leaf* (*insert t s*))))

We define a third invariant that ensures that the structure of the trie is consistent with the arity of symbols, and that edges decorated with a symbol of arity 0 lead to a leaf. This property is needed in many proofs to eliminate vacuous cases.

inductive *well-structured* :: $('f,\ 'v)$ *trie* $\Rightarrow$ *bool*
and *well-structured-map-entry* :: *'f symbol* $\Rightarrow$ (*nat*, $('f,\ 'v)$ *trie*) *fmap* $\Rightarrow$ *bool*
where
  ($\forall$ *s*. *s* $\in$ *fmdom'* *m* $\longrightarrow$ *well-structured-map-entry s* (*m* \$\$! *s*)) $\Longrightarrow$
    well-structured (*SymbolNode m*)
| *fmdom'* *m* = {*0*} $\Longrightarrow$ ($\forall$ *t*. *t* $\in$ *fmran'* *m* $\longrightarrow$ ($\exists$ *s*. *t* = *Leaf s*)) $\Longrightarrow$

$$
\begin{array}{l}
\quad \textit{well-structured-map-entry Star } m \\
\mid \textit{fmdom}' \ m = \{0\} \Longrightarrow (\forall\, t.\ t \in \textit{fmran}' \ m \longrightarrow (\exists\, s.\ t = \textit{Leaf } s)) \Longrightarrow \\
\quad \textit{well-structured-map-entry } (F\ f\ 0)\ m \\
\mid \textit{arity} \neq 0 \Longrightarrow \textit{fmdom}' \ m = \{..< \textit{arity}\} \Longrightarrow \\
\quad (\forall\, t.\ t \in \textit{fmran}' \ m \longrightarrow \textit{well-structured } t) \Longrightarrow \\
\quad\quad \textit{well-structured-map-entry } (F\ f\ \textit{arity})\ m
\end{array}
$$

Lastly, we define a fourth invariant that ensures that the trie contains no empty leaves or childless nodes, except possibly the root itself. While not needed for functional correctness, this is a desirable property that should be preserved by maintenance operations, in particular term removal.

$$
\begin{array}{l}
\textbf{inductive } \textit{minimal-storage} :: \ ('f,'v)\ \textit{trie} \Rightarrow \textit{bool} \textbf{ where} \\
\quad x \neq \{\} \Longrightarrow \textit{minimal-storage } (\textit{Leaf } x) \\
\mid (\forall\, t \in \textit{fmran}' \ m.\ \forall\, r \in \textit{fmran}' \ t.\ \textit{minimal-storage } r \wedge r \neq \textit{SymbolNode fmempty}) \Longrightarrow \\
\quad \textit{minimal-storage } (\textit{SymbolNode } m)
\end{array}
$$

Proofs on tries require induction, however, properties *sound-storage* and *complete-storage* hold only for the root of the trie, and not for subtries. To address this, these invariants must be reformulated in a way that accounts for the position (given by a path) of a subtrie within the trie.

$$
\begin{array}{l}
\textbf{definition } \textit{sound-storage-from-position} :: ('f,\ 'v)\ \textit{trie} \Rightarrow 'f\ \textit{tpath} \Rightarrow \textit{bool} \textbf{ where} \\
\quad \textit{sound-storage-from-position tr } p = \\
(\forall\, t\ p2\ s.\ \textit{subtrie-at-path tr } p2\ (\textit{Leaf } (\textit{insert } t\ s)) \longrightarrow \textit{path-of-term } t\ (p\ @\ p2))
\end{array}
$$

$$
\begin{array}{l}
\textbf{definition } \textit{complete-storage-from-position} :: ('f,\ 'v)\ \textit{trie} \Rightarrow 'f\ \textit{tpath} \Rightarrow \textit{bool} \textbf{ where} \\
\quad \textit{complete-storage-from-position tr } p = \\
\quad\quad (\forall\, t \in \textit{to-set tr}.\ \forall\, p2.\ \textit{path-of-term } t\ (p\ @\ p2) \longrightarrow \\
\quad\quad\quad (\exists\, s.\ \textit{subtrie-at-path tr } p2\ (\textit{Leaf } (\textit{insert } t\ s))))
\end{array}
$$

where @ is the *append* operator on lists. The properties *sound-storage* and *complete-storage* of the root are simply special cases of the above, from the path/position []. We can easily verify that, given the correct position, these properties hold recursively for subtries, e.g.:

$$
\begin{array}{l}
\textbf{lemma } \textit{complete-storage-from-position-subtrie}: \\
\quad \textbf{assumes } \textit{complete-storage-from-position } (\textit{SymbolNode m1})\ p \\
\quad\quad \textbf{and } m1\ \$\$\ f = \textit{Some } m2 \textbf{ and } m2\ \$\$\ i = \textit{Some tr} \\
\quad \textbf{shows } \textit{complete-storage-from-position tr } (p\ @\ [(f,\ i)])
\end{array}
$$

With these invariants, we can define a new type for valid tries:

$$
\begin{array}{l}
\textbf{typedef } ('f,\ 'v)\ \textit{index} = \{\ \textit{tr} :: ('f,\ 'v)\ \textit{trie}.\ \textit{well-structured tr} \wedge \textit{sound-storage tr} \\
\qquad\qquad\qquad\qquad \wedge\ \textit{complete-storage tr} \wedge \textit{minimal-storage tr}\ \}
\end{array}
$$

Note that the root of such a trie must be a *SymbolNode*. In particular the empty trie is *SymbolNode fmempty* rather than an empty leaf.

4.2 Maintenance Operations

We have formalized and proved the correctness of a term insertion function and a term deletion function. Both functions traverse the trie from the root node to the leaves, following the paths of the term to insert/delete, updating or creating/deleting nodes along the path as needed.

Let us briefly define some helper functions used in the insertion function. *insert-leaf tr t* is defined only if *tr* is a *Leaf*, in which case it adds the term *t* to the terms contained in that leaf. *update-entry* is a higher-order function that modifies a map in the following way: given a map *m*, a key *k*, a function *f* and a default value *v*, *update-entry m k f v* outputs the map that associates *k* to *Some* (*f* (*m* \$\$! *k*)) if *k* is defined in *m*, and to *Some* (*f v*) otherwise, and associates other keys *k′* to *m* \$\$ *k′*. *update-entries* is similar but operates on a list of keys rather than a single key (the update function *f* also take the key as parameter). We can now define the auxiliary recursive function *insert-aux*, such that *insert-aux tr t1 t2* is a trie where the term *t2* is present in all the leaves found on the paths of term *t1*.

fun *insert-aux* :: (′*f*, ′*v*) *trie* ⇒ (′*f*, ′*v*) *term* ⇒ (′*f*, ′*v*) *term* ⇒ (′*f*, ′*v*) *trie* **where**
insert-aux (*Leaf s*) *t t0* = *undefined*
| *insert-aux* (*SymbolNode m*) (*Var x*) *t0* =
SymbolNode (*update-entry m Star* (λ*m2*. *update-entry m2 0* (λ*tr*. *insert-leaf tr t0*) (*Leaf* {})) *fmempty*)
| *insert-aux* (*SymbolNode m*) (*Fun f* []) *t0* =
SymbolNode (*update-entry m* (*F f 0*) (λ*m2*. *update-entry m2 0* (λ*tr*. *insert-leaf tr t0*) (*Leaf* {})) *fmempty*)
| *insert-aux* (*SymbolNode m*) (*Fun f fargs*) *t0* =
SymbolNode (*update-entry m* (*F f* (*length fargs*))
(λ*m2*. *update-entries m2* [*0*..<*length fargs*] (λ*i tr*. *insert-aux tr* (*fargs* ! *i*) *t0*) *empty-trie*) {\$\$})

This auxiliary function is called by the main insertion function

definition *insert-term* :: (′*f*, ′*v*) *trie* ⇒ (′*f*, ′*v*) *term* ⇒ (′*f*, ′*v*) *trie* **where**
insert-term tr t = *insert-aux tr t t*

which we lift to the type of valid tries

lift-definition *index-insert* :: (′*f*, ′*v*) *index* ⇒ (′*f*, ′*v*) *term* ⇒ (′*f*, ′*v*) *index* **is** *insert-term*

The type of this lifted function indicates the preservation of the four invariants, which we need to prove. The proof of preservation of *well-structured* and *minimal-storage* are direct, but for *sound-storage* and *complete-storage*, we need to generalize, e.g., for completeness, with the following lemma:

lemma *insert-preserves-complete*:
assumes *well-structured tr* **and** *complete-storage-from-position tr p*
and *subterm-at-path t p t′*
shows *complete-storage-from-position* (*insert-aux tr t′ t*) *p*

Finally we specify the behavior of *insert-term* w.r.t. the content of the trie, using the function *to-set* and set insertion:

lift-definition *index-to-set* :: $('f, 'v)$ *index* $\Rightarrow$ $('f, 'v)$ *term set* **is** *to-set*

theorem *index-insert-spec*: *index-to-set* (*index-insert tr t*) = *insert t* (*index-to-set tr*)

Here, for the proof of membership of the newly inserted term, it is enough to focus on a single arbitrary path of that term.

The term removal function, not shown here, works in the same way as insertion, however, it also ensures that leaves and nodes are deleted from the trie if they are emptied by the removal. We prove that it preserves the invariants, and we specify its behavior w.r.t. the content of the trie, as we did for term insertion.

4.3 Retrieval Operations

We have formalized and proved the correctness of four retrieval operations, namely retrieval of unifications, generalizations, instances and variants. We only present here the formalization of the *retrieve-unifications* function, but similar ideas are used for the other functions.

We first define the function *var-terms* that returns all the terms in the leaf under a *Star* edge, if any. We use the partial lookup function \$\$!, since in a *well-structured* node, the key 0 must be defined for the map *m2*.

fun *var-terms* :: $(('f\ symbol, (nat, ('f, 'v)\ trie)\ fmap)\ fmap) \Rightarrow ('f, 'v)$ *term set* **where**
  var-terms m = (*case m* \$\$ *Star of None* $\Rightarrow$ {} | *Some m2* $\Rightarrow$ *to-set* (*m2* \$\$! *0*))

Then we can define the main function for the retrieval of terms that are unifiable with a given query term:

fun *retrieve-unifications* :: $('f, 'v)$ *trie* $\Rightarrow$ $('f, 'v)$ *term* $\Rightarrow$ $('f, 'v)$ *term set* **where**
  retrieve-unifications (*Leaf ts*) *t* = *undefined*
| *retrieve-unifications* (*SymbolNode m*) (*Var x*) = *to-set* (*SymbolNode m*)
| *retrieve-unifications* (*SymbolNode m*) (*Fun f fargs*) =
    var-terms m $\cup$
    (*case m* \$\$ (*F f* (*length fargs*)) *of*
      None $\Rightarrow$ {}
    | *Some m2* $\Rightarrow$ *if fargs* = [] *then to-set* (*m2* \$\$! *0*)
                else $\bigcap$ *i* $\in$ *fmdom′ m2*. *retrieve-unifications* (*m2* \$\$! *i*) (*fargs* ! *i*))

Finally we can lift the function to the type *index* and prove its specification:

lift-definition *index-retrieve-unifications* ::
  $('f, 'v)$ *index* $\Rightarrow$ $('f, 'v)$ *term* $\Rightarrow$ $('f, 'v)$ *term set* **is** *retrieve-unifications*

theorem *index-retrieve-unifications-spec*:
  index-retrieve-unifications tr t1 = {*t2*. *t2* $\in$ *index-to-set tr* $\wedge$ *linear-unification t1 t2*}

For this, we need to show that the answer set only contains elements from *to-set* that are linear unifications of the query term (soundness), and that all linear unifications in *to-set* are in the answer set (completeness). Once again this is achieved through more general inductive lemmas, e.g., for completeness:

lemma *retrieve-unifications-complete*:
assumes *well-structured tr* **and** *complete-storage-from-position tr p*
and *sound-storage-from-position tr p*
and *linear-unification t t′* **and** $t'' \in$ *to-set tr* **and** *subterm-at-path t′′ p t′*
shows $t'' \in$ *retrieve-unifications tr t*

5 Conclusion

We have presented an Isabelle/HOL formalization of path indexing, the first verified implementation of a term indexing data structure. In future work, we plan to formalize other term indexing techniques, starting with discrimination trees. While Isabelle allows code extraction, to obtain efficient implementations, we will need to verify imperative code with mutable data structures, as opposed to the purely functional implementation presented here. Refinement techniques exist to derive, from functional Isabelle programs, Imperative HOL code [4] or even an LLVM intermediate representation [5].

Disclosure of Interests The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Blanchette, J.C., Fleury, M., Lammich, P., Weidenbach, C.: A verified SAT solver framework with learn, forget, restart, and incrementality. Journal of automated reasoning **61**, 333–365 (2018)
2. Fleury, M., Blanchette, J.C., Lammich, P.: A verified SAT solver with watched literals using imperative HOL. In: Proceedings of the 7th ACM SIGPLAN International Conference on Certified Programs and Proofs. pp. 158–171 (2018)
3. Graf, P.: Path indexing for term retrieval. Tech. rep., Max-Planck-Institut für Informatik (1992)
4. Lammich, P.: Automatic data refinement. In: International Conference on Interactive Theorem Proving. pp. 84–99. Springer (2013)
5. Lammich, P.: Generating verified LLVM from Isabelle/HOL. In: 10th International Conference on Interactive Theorem Proving (ITP 2019). Schloss Dagstuhl-Leibniz-Zentrum für Informatik (2019)
6. McCune, W.: Experiments with discrimination-tree indexing and path indexing for term retrieval. Journal of automated reasoning **9**(2), 147–167 (1992)
7. Nipkow, T., Klein, G.: Concrete semantics with Isabelle/HOL. Springer (2014)
8. Nipkow, T., Paulson, L.C., Wenzel, M.: Isabelle/HOL: a proof assistant for higher-order logic, LNCS, vol. 2283. Springer (2002)
9. Ramesh, R., Ramakrishnan, I., Warren, D.S.: Automata-driven indexing of prolog clauses. In: Proceedings of the 17th ACM SIGPLAN-SIGACT symposium on Principles of programming languages. pp. 281–291 (1989)
10. Schlichtkrull, A., Blanchette, J., Traytel, D., Waldmann, U.: Formalizing Bachmair and Ganzinger's ordered resolution prover. Journal of Automated Reasoning **64**(7), 1169–1195 (2020)

11. Sekar, R., Ramakrishnan, I., Voronkov, A.: Term indexing. In: Handbook of Automated Reasoning. pp. 1853–1964. Elsevier (2001)
12. Stickel, M.E.: The path-indexing method for indexing terms. SRI International. Artificial Intelligence Center (1989)
13. Waldmann, U., Tourret, S., Robillard, S., Blanchette, J.: A comprehensive framework for saturation theorem proving. Journal of Automated Reasoning pp. 1–41 (2022)